DRIFA-Net Adaptation — Source Material for Research Report

Internal reference document. Compiled for report drafting — not itself the report. Every figure below is either directly measured from this project's own runs, or explicitly marked as unverified/pending/estimated. Do not present estimated figures as measured results in the final report.

1. Original Work Being Adapted

Item	Detail
Paper	"Multimodal Fusion Learning with Dual Attention for Medical Imaging" — arXiv:2412.01248
Authors	Joy Dhar, Nayyar Zaidi, Maryam Haghighat, Puneet Goyal, Sudipta Roy, Azadeh Alavi, Vikas Kumar
Proposed method	DRIFA — Dual Robust Information Fusion Attention. Two attention modules: a multi-branch fusion attention module (MFA) and a multimodal information fusion attention module (MIFA), plus Monte Carlo Dropout for uncertainty estimation.
Original experimental scope (per abstract)	Five publicly available datasets across diverse modalities (dermoscopy, pap smear, MRI, CT-scan are named as modality types in the abstract; exact dataset identities and per-dataset metrics are **not** stated in the abstract and were not independently verified in this session — pull exact table values directly from the paper's PDF/results tables for the report, do not cite figures from this document as the paper's own results).
Reference code repository	Referenced in project materials as `github.com/misti1203/DRIFA-Net` — this is this project's own working repository (see §7), not confirmed to be the original authors' repository. Verify and cite the authors' own repository link from the paper itself for the report.

> This project does **not** reproduce the original paper's experimental protocol. It adapts the architecture to a different, smaller pair of datasets. Frame this explicitly as an adaptation in the report, not a reproduction — see §11.

2. Why the Datasets Were Substituted

The notebook being adapted (`Code.ipynb`) originally expected preprocessed NumPy arrays for one of its dataset branches (`X_train_HAM10000_ISIC_2018.npy`, etc.) that were not available in this environment, and its second dataset was **SIPaKMeD** (cervical cytology), which was also unavailable. Rather than reproduce the original five-dataset, multi-modality experimental protocol, the project scope was narrowed to two datasets that were obtainable and represent a genuine dual-modality classification problem: **HAM10000** (kept, but loaded from raw images + metadata instead of missing `.npy` files) and **Brain Tumor MRI** (substituted for SIPaKMeD).

3. Adapted Datasets

3.1 Brain Tumor MRI (branch/input 1)

Property	Value
Classes (4)	glioma=0, meningioma=1, notumor=2, pituitary=3
Source structure	`Dataset/brain tumor/Training/{class}` and `Testing/{class}` folders, used directly (no extra train/test split needed)
Raw verified shapes	X_train=(5600,128,128,3), y_train=(5600,4), X_test=(1600,128,128,3), y_test=(1600,4)
Preprocessing	Resize to 128x128 RGB, normalize /255
Augmentation	+20° rotation and (5,5) pixel translation applied to each of the 5,600 training images (11,200 possible augmented samples); 4,773 of these were selected and concatenated with the 5,600 originals → **10,373** Brain MRI training samples
Test-set duplication note	A separate copy of the test set was tripled and reduced to 2,003 samples specifically to index-align with HAM10000's 2,003-sample test set for one of the two evaluation passes used in this notebook (see §9). The original, non-duplicated 1,600-sample test set is also retained and used in the other evaluation pass.

3.2 HAM10000 (branch/input 2)

Property	Value
Classes (7)	akiec=0, bcc=1, bkl=2, df=3, mel=4, nv=5, vasc=6
Source	`HAM10000_metadata.csv` + JPGs across two image-part directories
Split	80/20 stratified split by class (`train_test_split`, `random_state=42`)
Verified shapes	X_train=(8012,128,128,3), y_train=(8012,7), X_test=(2003,128,128,3), y_test=(2003,7)
Preprocessing	Resize to 128x128 RGB, normalize /255. No augmentation applied to this branch.

3.3 HAM10000 class distribution — measured directly from `HAM10000_metadata.csv`

Class	Count	% of full dataset
nv	6,705	66.95%
mel	1,113	11.11%

bkl	1,099	10.97%
bcc	514	5.13%
akiec	327	3.27%
vasc	142	1.42%
df	115	1.15%

Largest:smallest class ratio (nv:df) = **58:1**. This severe imbalance is the central diagnosed problem addressed by the optimization in §10.

3.4 Sample-count alignment (multi-input training constraint)

Keras multi-input models require every input/output pair in a single `.fit()` call to share the same sample count along the batch axis. After augmentation, Brain MRI (10,373) and HAM10000 (8,012) training sets had different lengths. Alignment: Brain MRI training data is shuffled and truncated to HAM10000's length (8,012), producing two length-matched training sets without permanently discarding the augmented pool (truncation, not re-augmentation, so this can be re-run with a different random shuffle). This alignment step, and the memory-duplication risk associated with earlier versions of it, is discussed further in §6.

4. Model Architecture

The architecture is retained largely unchanged from the original repository. Component-by-component:

Module	Status	Role
MFA (`DeeperAttentionLayer1` / `DeeperGlobalLocalAttentionLayer1`)	Unchanged	Per-branch attention combining Hierarchical Information Fusion Attention (global avg/max-pooled, multi-stage) and Channel-wise Local Information Attention (CLIA), fused via a learned global/local scale and sigmoid gate. Applied independently within each branch.
MIFA (`DeeperAttentionLayer` / `DeeperGlobalLocalAttentionLayer`)	Unchanged	Cross-branch fusion attention. Computes Multimodal Global Information Fusion Attention (MGIFA: joint min/avg/max pooling across **both** branches) and Multimodal Local Information Fusion Attention (MLIFA), producing one shared sigmoid attention map applied to *both* branches, each scaled by its own learned `alpha1`/`alpha2` parameter.
RGSA	Unchanged	Residual block: Conv2D → MFA → BatchNorm → ReLU → Conv2D → MFA → BatchNorm → residual add → ReLU. Used repeatedly at each channel-width stage.
`residual_GLC_branch1`	Unchanged	Full dual-branch backbone (see §4.1)
Classifier heads	Modified	Original: Dense(5, softmax) + Dense(7, softmax). Adapted: **Dense(4, softmax)** for Brain MRI + Dense(7, softmax) for HAM10000 (unchanged class count).

4.1 Verified data flow (read directly from the model-build code, not the paper's schematic)

Important correction to an early assumption in this project's planning: MIFA is not applied once after two independent MFA-only branches. It is interleaved with RGSA **eight times** throughout the backbone, once after each channel-width stage:

```
Input1 (Brain MRI, 128x128x3) Input2 (HAM10000, 128x128x3) -> Conv2D(64,7x7,s2) -> MFA -> BN ->
ReLU -> MaxPool (mirrored on input2) -> RGSA(64) -> Dropout(MCD) -> MFA | RGSA(64) ->
Dropout(MCD) -> MFA -> MIFA #1 (fuses both branches, 64ch) -> RGSA(64) -> Dropout(MCD) -> MFA |
RGSA(64) -> Dropout(MCD) -> MFA -> MIFA #2 (64ch) -> RGSA(128, stride2) -> Dropout -> MFA |
RGSA(128, stride2) -> Dropout -> MFA -> MIFA #3 (128ch) -> RGSA(128) -> Dropout -> MFA |
RGSA(128) -> Dropout -> MFA -> MIFA #4 (128ch) -> RGSA(256, stride2) -> Dropout -> MFA |
RGSA(256, stride2) -> Dropout -> MFA -> MIFA #5 (256ch) -> RGSA(256) -> Dropout -> MFA |
RGSA(256) -> Dropout -> MFA -> MIFA #6 (256ch) -> RGSA(512, stride2) -> MFA | RGSA(512, stride2)
-> MFA -> MIFA #7 (512ch) -> RGSA(512) | RGSA(512) -> MIFA #8 (512ch) -> returns x1
```

```
(None,4,4,512), x2 (None,4,4,512) Concatenate([x1, x2], axis=-1) -> (None, 4, 4, 1024) *** no attention/weighting applied here today *** -> Dropout(0.25, training=True) [Monte Carlo Dropout, retained from original repo] -> GlobalAveragePooling2D() -> shared vector (None, 1024) -> Dense(4, softmax) "BrainMRI_output" -> Dense(7, softmax) "HAM10000_output"
```

Two facts follow directly from this that matter for §12 (the value-addition design):

- MIFA's `alpha1`/`alpha2` scale parameters are declared as `shape=(1,1,1,C)` — **global** learned weights, identical for every sample, not sample-adaptive, despite MIFA's attention map itself being input-dependent.
- The final `Concatenate([x1, x2])` — the single point that actually feeds both classification heads — applies **no attention or learned weighting whatsoever**. Both branches enter at full, fixed, equal weight. This is the one true "naive fusion" point in the whole network.

4.2 Model size

Metric	Value
Total parameters	53,883,843
Trainable parameters	53,864,643
Non-trainable parameters	19,200
Input resolution	128 x 128 x 3, both branches
Shared fused representation	(None, 1024)

5. Training Configuration

Setting	Value
Optimizer	Adam, learning_rate=0.001
Loss (both heads)	categorical_crossentropy
Loss weights	[0.5, 0.5] (`initial_gamma=0.5`) — fixed, equal, in the baseline
Callbacks	ModelCheckpoint(monitor='val_loss', save_best_only=True) → `best_DRIFA_brain_HAM.keras`; EarlyStopping(patience=100); ReduceLROnPlateau(factor=0.2, patience=5, min_lr=1e-5)
Intended epochs	200 (capped by EarlyStopping patience=100)
Validation	validation_split=0.2 (drawn from the aligned 8,012-sample training set)
Precision policy	`mixed_float16` global policy enabled
Batch size	32 (Keras default; not explicitly set) → 201 steps/epoch on the aligned training set

5.1 Hardware and measured throughput

Item	Value
GPU	NVIDIA T1000 (local), via `tensorflow-directml` plugin, TensorFlow 2.10.1 (not CUDA)
Devices detected	Two DirectML PluggableDevices (GPU:0, GPU:1)
Forward-only throughput (`.evaluate()`)	~245 ms/step
Real training throughput (forward+backward, measured on the in-progress run in §10)	~3.4 s/step → **~11.5 minutes/epoch measured** (201 steps/epoch)

> The ~14x gap between forward-only and train-step throughput on this DirectML GPU is larger than the ~2x typically seen on CUDA hardware, and was not anticipated early in the project — worth stating explicitly in the report as a hardware-driven constraint on how much experimentation was feasible (see §13).

6. Development History: Colab Crash → Local T1000 Migration

The project originally targeted Google Colab. Training runs there repeatedly caused full process crashes (not clean Python out-of-memory exceptions) at the point training began. Root cause: 128x128x3 float32 arrays were simultaneously duplicated across original, augmented, and sample-aligned copies of both datasets, plus TensorFlow/model memory — exceeding available RAM even though the GPU (T4) itself was correctly selected and had headroom. GPU choice does not address CPU/RAM-side array duplication.

The project was migrated to a local machine with an NVIDIA T1000. The memory-duplication issue did not recur locally; the alignment step (§3.4) was rewritten to avoid creating additional large NumPy copies beyond the shuffle/truncate operation already described.

7. Execution Environment and Repository

Item	Detail
Working notebook	`Code_v2.ipynb` (adapted from the original repository's `Code.ipynb`)
Execution method	`papermill Code_v2.ipynb Code_v2.ipynb --log-output --kernel python3`, run from a local Python virtual environment, logged to `execution_log.txt.out/.err`
Git repository	`github.com/misti1203/DRIFA-Net` (origin remote of this working copy)
Repository status at time of writing	Local commit `1a2e391` ("Add T1000-adapted DRIFA-Net notebook and updated status docs") created, containing `Code_v2.ipynb`, both status documents, `requirements.txt`, and a new `.gitignore` (excludes the dataset folder, virtual environment, `.keras` checkpoints, and log files from version control, as these are large binaries/generated artifacts unsuitable for git). **Push to the remote is currently blocked** — the authenticated account does not have write permission to this repository; unresolved as of this writing and needs to be fixed outside of this document's scope before the code is actually on GitHub.

8. Evaluation Methodology

Predictions from both softmax heads are converted from one-hot/probability vectors to class labels via `np.argmax` (an earlier version of the evaluation code incorrectly used binary thresholding (`y_pred >= 0.5`), which is invalid for multiclass softmax outputs — this was identified and corrected). Metrics reported: accuracy, and macro-averaged precision/recall/F1 (macro averaging — unweighted mean across classes — is what actually exposes the HAM10000 minority-class problem in §9; accuracy alone hides it). A side-by-side confusion matrix (one per branch) was added as a dedicated notebook cell using `sklearn.metrics.ConfusionMatrixDisplay`.

Note on test-set pairing: because the model has two inputs, evaluation is run with two different Brain-MRI/HAM10000 test-set pairings depending on which one is being index-aligned to the other's length (2,003-sample alignment vs. 1,600-sample original) — both pairings are present in the notebook; the headline baseline numbers in §9 come from the pairing using the original, non-duplicated 1,600-sample Brain MRI test set paired with a 1,600-sample HAM10000 subset.

9. Results — Stage 1: Adapted Baseline (post-training, "epoch-26" checkpoint)

Full training was run under the configuration in §5. A second, further training pass was deliberately stopped and the model was rolled back to the checkpoint saved at what the project refers to as "epoch 26" (the last `ModelCheckpoint`-saved improvement before the run was manually halted), used as the final baseline model. This baseline uses the original fixed `loss_weights=[0.5,0.5]` and no sample weighting.

Branch	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)
Brain MRI (4-class)	92.375%	92.74%	92.375%	92.18%
HAM10000 (7-class)	77.6875%	55.85%	51.39%	53.21%

Interpretation: Brain MRI is well-learned and balanced across metrics. HAM10000 shows a large gap between accuracy (77.7%) and macro recall/F1 (~51-53%) — the signature of a model that scores well by defaulting to the majority class (`nv`, 67% of the data — see §3.3) while performing poorly on minority classes. This is the finding that motivated the optimization in §10.

> Per-epoch training-curve data for this original 26-epoch baseline run itself was not preserved in this project's logs (only the final evaluation was captured) — the report should present §9's numbers as the single final-checkpoint result, not as part of an epoch-by-epoch trajectory, unless the original training log can still be recovered from elsewhere.

10. Results — Stage 2: Class-Balanced Optimization (in progress)

10.1 Motivation

Diagnosed cause of HAM10000's low macro recall/F1 (§9): severe class imbalance (§3.3), a 58:1 ratio between the largest and smallest class. The model has no incentive under plain `categorical_crossentropy` to learn the rare classes well, since predicting the majority class is already a low-loss strategy for most samples.

10.2 Method

Class-balanced per-sample weighting applied to the HAM10000 loss only (Brain MRI is already class-balanced by construction of its dataset, so it receives uniform weight 1.0 per sample). Weights computed via `sklearn.utils.class_weight.compute_class_weight('balanced', ...)`, which sets each class's weight inversely

proportional to its frequency:

```
y_train_h_labels = np.argmax(y_train_h, axis=1) class_weights_ham =
compute_class_weight('balanced', classes=np.arange(7), y=y_train_h_labels) sample_weight_ham =
class_weights_ham[y_train_h_labels] sample_weight_brain = np.ones(len(y_train_s)) model.fit(
x=[X_train_s, X_train_h], y=[y_train_s, y_train_h], sample_weight=[sample_weight_brain,
sample_weight_ham], epochs=20, validation_split=0.2, shuffle=True, callbacks=callbacks )
```

Class	Computed weight
nv	≈0.21x
mel	≈1.29x
bkl	≈1.30x
bcc	≈2.78x
akiec	≈4.37x
vasc	≈10.0x
df	≈12.4x

Training resumes from the existing baseline checkpoint (not from scratch) for 20 epochs, and saves to a new file (`best_model_class_weighted.keras`) so the original baseline artifact (`best_model_ever.keras`) is preserved for before/after comparison rather than overwritten.

10.3 Status: IN PROGRESS — NOT YET COMPLETE

As of this writing, training is in epoch 4 of 20. Observed so far:

Epoch	val_loss	Checkpointed?
1	1.20016	Yes (improved from inf)
2	0.68784	Yes (improved)
3	(no improvement over 0.68784)	No

At the measured ~11.5 min/epoch (§5.1), full completion of all 20 epochs is expected to take roughly 3.5-4 hours from the start of this run. **Final accuracy/precision/recall/F1 numbers for this stage are not yet available and must not be fabricated in the report** — re-run the evaluation cells once training finishes and insert the real numbers before submission.

10.4 Expected direction of the result (estimate, explicitly not a measured result)

Metric (HAM10000)	Baseline (§9)	Estimated range post-optimization
Accuracy	77.7%	~68-75% (a drop is the expected, correct outcome — see rationale below)
Macro Recall	51.4%	~60-70%

Macro Precision	55.9%	~58-65%
Macro F1	53.2%	~60-67%

Why accuracy is expected to drop while recall/F1 rise: the baseline's accuracy is inflated by defaulting to the majority class (nv). Once minority classes carry 5-12x more gradient weight, the model will start actually predicting rare classes sometimes — correctly classifying some of them (raising recall/F1) while also occasionally misclassifying borderline majority-class cases it previously got "for free" (lowering raw accuracy). This trade-off is the intended, correct behavior of class balancing for an imbalanced medical classification task, not a defect.

11. Results — Stage 3: Architecture-Level Value Addition

11.1 Status: NOT YET IMPLEMENTED — DESIGN ONLY

This section documents a fully-scoped, not-yet-executed architectural addition, planned to be evaluated on top of the Stage 2 (class-balanced) checkpoint once it finishes training, so its own contribution can be isolated from the class-balancing gain rather than conflated with it.

11.2 Motivation and where it fits

As established in §4.1, the one genuinely unweighted point in the entire architecture is the final `Concatenate([x1, x2])` immediately before `GlobalAveragePooling2D` and the two classification heads — both branches enter at fixed, equal weight, with no learned adaptivity at all, unlike every earlier stage of the network (which is saturated with MFA/MIFA attention). This is the proposed insertion point.

11.3 Proposed module: Adaptive Fusion Gate (AFG)

For each sample, generate one scalar "branch contribution" score per branch from pooled features, normalize competitively via softmax, and use the result to reweight each branch before concatenation:

```
z1 = Dense(1)(Dense(16, activation='relu')(GlobalAveragePooling2D()(x1))) z2 = Dense(1)(Dense(16, activation='relu')(GlobalAveragePooling2D()(x2))) alpha, beta = softmax([z1, z2]) # alpha + beta = 1, per sample
```

Safety-constrained (zero-init residual) formulation — required because the naive version above changes behavior at initialization (softmax gives alpha=beta≈0.5, which *halves* both branches' feature magnitude relative to the current baseline's implicit full-weight, i.e. "1+1", concatenation — a real regression risk before any training even happens). The corrected formulation introduces one additional trainable scalar `scale`, initialized to 0:

```
x1' = x1 * (1 + scale * (2*alpha - 1)) # at scale=0: x1' = x1 exactly x2' = x2 * (1 + scale * (2*beta - 1)) # at scale=0: x2' = x2 exactly Concatenate([x1', x2']) # replaces the current Concatenate([x1, x2])
```

At initialization (`scale=0`), the model is provably, mathematically identical to the current baseline — this should be explicitly verified via a smoke test (feed identical input through both the original and modified model with `scale` forced to 0 and confirm outputs match to floating-point tolerance) before any training run. As training proceeds, `scale` only moves away from 0 if doing so reduces the loss.

11.4 Novelty assessment against the paper's existing mechanisms

Question	Answer
Does MIFA already do this?	No. MIFA's `alpha1`/`alpha2` scale parameters (§4.1) are global, not sample-specific, and MIFA is applied 8 times deep inside the backbone, not at the final fusion point feeding the classifier heads.
What does MIFA learn?	A shared, input-dependent spatial/channel attention map (from joint min/avg/max statistics of both branches) applied identically to both branches at 8 intermediate stages, each branch additionally scaled by its own *global* (not per-sample) learned parameter.

What would AFG learn?	A single competitive, sample-specific scalar weighting between the two branches, applied once, at the final fusion point that currently has zero attention of any kind.
Is the difference sufficient to call this a methodological addition?	Yes, with correct framing (§11.5) — it is additive to, not a replacement for, MIFA, and it operates on a part of the network MIFA does not touch.

11.5 Required honest framing / limitations for the report

- **Do not describe this as "the model learns which imaging modality is more reliable for the patient."** Brain MRI sample *i* and HAM10000 sample *i* are index-aligned only to satisfy Keras's multi-input batching (§3.4) — they are not two modalities of the same patient. Use "branch contribution" or "sample-specific feature contribution" instead of "modality reliability" throughout.
- **Shared-vector trade-off**: the fused vector feeds *both* classification heads. If the gate learns to favor the Brain MRI branch for a given sample, it is simultaneously reducing HAM10000's share of that same forward pass, and vice versa — a structural trade-off between the two tasks that the zero-init safety design does not remove (it only guarantees a safe *starting point*, not safe *training dynamics*). Report both branches' metrics separately, not just an aggregate, since one task's metric moving up while the other moves down is a plausible, non-broken outcome.
- **No guaranteed accuracy improvement.** The zero-init design guarantees the model cannot be worse than baseline *at initialization*; it does not guarantee the *trained* result improves on baseline. Report results as measured, hedged appropriately.

11.6 Estimated cost

Metric	Value
Added parameters	≈16,500 (GAP + Dense(16) + Dense(1) per branch, plus one scalar) vs. 53,883,843 total — ≈0.03% increase
Added inference compute	Two small dense layers operating on an already-pooled 512-length vector per branch; negligible next to the convolutional backbone

11.7 Alternatives considered and why they were not selected

Candidate	Summary	Why not chosen

Dynamic Weight Averaging (DWA)	Adjusts the two *task loss* weights `[w_brain, w_HAM]` per epoch based on each task's relative loss-improvement rate; naturally starts at [0.5,0.5] (identity-safe) since no history exists for the first two epochs.	Technically sound and arguably safer (zero inference-time cost, since it only affects training-time loss combination) — but it is also fundamentally a reweighting technique, the same category as the Stage 2 class-balancing fix already applied. Combining two loss-reweighting techniques risks reading as one idea done twice, rather than two distinct contributions spanning different levels of the pipeline. AFG was chosen instead specifically to diversify the overall contribution (data/loss-level fix + architecture-level fix).
Squeeze-and-Excitation (SE) channel recalibration on the fused vector	Standard channel-attention block (Hu et al., 2018) applied to the 1024-channel fused representation.	Lower risk than AFG (no branch-competition/shared-vector trade-off), but a well-established, widely-used technique — weaker as a presentable "contribution" than a purpose-built gate, and does not address the specific "no weighting at the final fusion point" gap as directly.
Learned attention pooling (replacing plain GlobalAveragePooling2D)	1x1 conv + softmax spatial attention over the 4x4 feature grid, weighted sum instead of uniform average.	Reasonable alternative; not pursued due to time constraints once AFG was selected as the primary candidate.

12. Threats to Validity / Required Disclosures

- **Adaptation, not reproduction.** The original paper's protocol uses five datasets across multiple modality pairs; this project uses two (Brain Tumor MRI + HAM10000, replacing SIPaKMeD). Dataset statistics, class counts, and the interpretation of "multimodal fusion" all differ from the original protocol as a result. State this plainly rather than imply a validated reproduction of the paper's reported results.
- **Unpaired dataset construction.** As discussed in §3.4 and §11.5, the two branches' samples are index-paired for training mechanics only, not biologically/clinically paired. Any language implying per-patient cross-modality reasoning should be avoided project-wide, not just for §11.
- **Hardware-constrained experimental scope.** The measured ~11.5 min/epoch training throughput (§5.1, far slower than initially estimated from forward-only benchmarking) genuinely limited how many epochs and how many comparative experiments were feasible within the project's available time. This is a real, disclosed constraint, not an excuse — state it as such, and note where results would benefit from longer/larger-scale validation on stronger hardware as future work.
- **Original paper's own reported metrics were not independently verified in this session** (§1) — the abstract does not state them; pull exact figures from the paper's results tables directly for any comparison claims in the report.

13. Suggested Report Structure

1. Introduction & motivation (cite DRIFA-Net, arXiv:2412.01248; state the adaptation scope from the start)
2. Related work (DRIFA-Net's MFA/MIFA mechanisms; briefly, class-imbalance handling literature; multi-task loss weighting literature for the DWA-vs-AFG discussion in §11.7)
3. Datasets & preprocessing (§3)
4. Architecture (§4) — include the corrected 8x-MIFA data-flow diagram from §4.1, not a simplified one-shot-fusion diagram
5. Experimental setup (§5, §7, §8)
6. Results: baseline (§9) → class-balanced optimization (§10) → AFG value addition (§11), each with its own metrics table
7. Discussion: why accuracy fell while recall/F1 rose in §10 (this is a finding worth explaining well, not hiding); AFG's shared-vector trade-off discussion from §11.5
8. Limitations (§12)
9. Conclusion & future work (larger-scale AFG variants, DWA as a complementary future direction, GradNorm, longer training on stronger hardware)

> Suggested contribution statement template (fill in only once §10 and §11 have real, measured numbers): "We adapt DRIFA-Net (Dhar et al., arXiv:2412.01248) to a Brain Tumor MRI + HAM10000 dual-branch setting, diagnose a severe class-imbalance-driven precision/recall gap in the HAM10000 branch via confusion-matrix analysis, address it with class-balanced loss weighting, and additionally propose a lightweight, safety-constrained Adaptive Fusion Gate at the network's one previously-unweighted fusion point, evaluated under a zero-initialization guarantee that it cannot regress baseline behavior at initialization."